

For this notebook: app_OpenAI_chatbot,

This code builds a **simple Q&A chatbot web app** using **Streamlit + OpenAI (via LangChain)**. I'll explain it clearly in sections so you understand what each part does.

✓ 1. Imports and setup

```
Python
import streamlit as st
import openai
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
import os
from dotenv import load_dotenv
```

What it means:

- streamlit → Used to create a web app UI
 - openai → For calling OpenAI API
 - ChatOpenAI → LangChain wrapper around OpenAI models
 - StrOutputParser → Converts model output into plain text
 - ChatPromptTemplate → Helps structure the prompt
 - dotenv → Loads environment variables from a .env file
-

✓ 2. Load environment variables

```
Python
load_dotenv()
```

- Reads secrets like API keys from .env file
-

✓ 3. LangSmith tracking (optional logging)

```
Python
os.environ["LANGCHAIN_API_KEY"]=os.getenv("LANGCHAIN_API_KEY")
os.environ["LANGCHAIN_TRACING_V2"]="true"
os.environ["LANGCHAIN_PROJECT"]="Simple Q&A Chatbot With OPENAI"
```

Purpose:

- Enables **LangChain tracing (debugging/logging)**
 - Tracks conversations in **LangSmith dashboard**
-

✓ 4. Prompt template

```
Python
prompt = ChatPromptTemplate.from_messages(prompt = Chat ("system","You are a helpful
massistant . Please repsonse to the user queries"),
("user","Question:{question}")
]
)
..
```

[

What it does:

Defines **how the AI should behave**

- System message → sets behavior
👉 "You are a helpful assistant..."
- User message → actual input
👉 "Question: {question}"

So if user asks:

What is Python?

The model actually receives:

System: You are a helpful assistant

User: Question: What is Python?

✓ 5. Core function: generate_response

Python

```
def generate_response(question,api_key,engine,temperature,max_tokens):  
    openai.api_key=api_key
```

```
    llm=ChatOpenAI(model=engine)  
    output_parser=StrOutputParser()  
    chain=prompt|llm|output_parser
```

```
    answer=chain.invoke({'question':question})  
    return answer
```

Step-by-step:

1. ✓ Set API key
2. ✓ Create model (ChatOpenAI)
3. ✓ Create output parser (text only)
4. ✓ Build a **chain**:
5. Prompt → LLM → Output parser
6. ✓ Run the chain with user question
7. ✓ Return response

👉 This is a **LangChain pipeline**

✓ 6. Streamlit UI

Python

```
st.title("Enhanced Q&A Chatbot With OpenAI")  
``
```

- Displays title of web app

✓ 7. Sidebar settings

Python

```
st.sidebar.title("Settings")
api_key=st.sidebar.text_input(...)
```

```
engine=st.sidebar.selectbox(...)
temperature=st.sidebar.slider(...)
max_tokens = st.sidebar.slider(...)
```

User can configure:

- ☒ API Key (required)
- ☒ Model (gpt-4o, gpt-4-turbo, etc.)
- ☒ Temperature (creativity level)
- ☒ Max tokens (response length)

☒ 8. User input area

Python

```
st.write("Go ahead and ask any question")
user_input = st.text_input("You:")
```

- User types their question here

☒ 9. Response logic

Python

```
if user_input and api_key:
    response = generate_response(...)
    st.write(response)
```

What happens:

- If user enters:
 - question ☒
 - API key ☒→ Bot generates answer

Edge cases:

Python

```
elif user_input:
    st.warning("Please enter the OpenAI API Key")
```

- If question entered but no API key → show warning

Python

```
else:
    st.write("Please provide the user input")
```

- If nothing entered → prompt user

☒ 🔄 Overall Flow

User opens app
↓
Enters API key + selects settings
↓
Types a question
↓
Prompt is created
↓
LangChain sends request to OpenAI
↓
AI generates response
↓
Response displayed in app

✓ Key Concepts Used

Concept	Meaning
Streamlit	Creates web UI
LangChain	Builds LLM pipelines
Prompt Template	Structures input to AI
Chain (`	`)
Temperature	Controls creativity
Model selection	Choose GPT version

✓ Small Issues in Code

- Typo: "massistant" → should be "assistant"
- "repsonse" typo
- temperature and max_tokens are collected but **NOT used in model**

🔗 Better version:

Python

```
llm = ChatOpenAI(model=engine, temperature=temperature, max_tokens=max_tokens)
```

✓ Final Summary (Simple)

🔗 This code creates a **custom ChatGPT-like app** where:

- You enter an API key
- Ask questions
- Choose model + creativity
- AI responds in real-time using OpenAI via LangChain

Note:

For this notebook: `app_ollama_chatbot`,

✓ One-line Summary

👉 Your code logic stays exactly the same — you just replace the OpenAI model with an Ollama model.

🔄 What Changes in `app_ollama_chatbot`

✅ Only ONE thing changes:

👉 The LLM (model provider)

Earlier	Now
OpenAI (ChatGPT models like gpt-4o)	Ollama (local models like llama3, mistral, etc.)